

Rapport — Mini-Projet A : Chiffrement de César & Enigma César

1. Structure du programme

Le code est découpé en **6 modules** aux responsabilités séparées, ce qui facilite la lecture, les tests et la collaboration via Git :

Module	Rôle
<code>cesar.py</code>	Chiffrement/déchiffrement César (1 clé entière)
<code>enigma.py</code>	Chiffrement/déchiffrement Enigma César (3 clés cycliques)
<code>brute_cesar.py</code>	Recherche de clé César (fréquences + dictionnaire)
<code>brute_enigma.py</code>	Recherche de clé Enigma (26^3 combinaisons)
<code>orchestrer.py</code>	Coordination des fonctions + normalisation du texte
<code>main.py</code>	Point d'entrée (<code>python main.py</code> , <code>argparse</code> + mode interactif)

Dépendances : `main` → `orchestrer` → {`cesar`, `enigma`, `brute_cesar`} ;
`brute_enigma` → {`brute_cesar`, `enigma`}. Les modules de chiffrement (`cesar`, `enigma`) n'importent rien du projet : ils sont **réutilisables et testables isolément**.

Choix de conception : la **normalisation** est centralisée dans `orchestrer.normaliser()` (minuscules + `unicodedata.NFKD` pour les accents) et appliquée systématiquement avant chiffrement et brute-force pour garantir la cohérence du scoring. Les **caractères hors alphabet** (ponctuation, espaces, chiffres) sont conservés tels quels pour préserver la lisibilité du texte chiffré. Côté clés, César accepte n'importe quel entier (positif/négatif, modulo 26 implicite) et Enigma exige un tuple de 3 entiers — saisie validée par `demander_cles_enigma()` qui redemande tant qu'elle est invalide. La lecture fichier utilise `with open(..., encoding="utf-8")` avec gestion d'`FileNotFoundError`.

2. Algorithmes

2.1 Chiffrement César — Décalage modulo 26 sur `string.ascii_lowercase` : $(\text{position} + \text{clé}) \% 26$. Complexité **$O(n)$** .

2.2 Chiffrement Enigma César — Les 3 clés sont appliquées **cycliquement** ($i \% 3$) en réutilisant `cesar.chiffrer()` lettre par lettre. Choix qui évite toute duplication entre les deux modes. Complexité **$O(n)$** .

2.3 Brute-force César — **stratégie hybride** combinée selon un seuil `SEUIL_MOTS = 15`. Sur un **texte long**, on score par **analyse de fréquences** (χ^2) : pour chacune des 26 clés, on déchiffre, on calcule l'histogramme des lettres et on le compare à la distribution française de référence (E $\approx$ 14,7 %, A $\approx$ 7,6 %, ...) ; la clé minimisant l'écart gagne. Sur un **texte court**, on

utilise un **dictionnaire de mots français** (`mots_francais.txt`, ~30 000 mots chargés dans un `set` pour des lookups en $O(1)$) et on garde la clé maximisant le nombre de mots reconnus. **Repli automatique** sur les fréquences si aucun mot n'est reconnu. *Justification du seuil* : sous 15 mots, l'histogramme devient trop bruité et donne des faux positifs ; le dictionnaire prend alors le relais.

2.4 Brute-force Enigma César — $26^3 = 17\,576$ combinaisons. Trois boucles imbriquées sur (`cle1`, `cle2`, `cle3`). Pour chaque triplet on déchiffre, puis on score avec **les mêmes fonctions** que pour César (`ressemble_au_francais`, `analyser_mots`), avec la même logique hybride.

3. Évaluation des performances

Machines : (1) MacBook Pro, Apple **M5 Pro** (15 cœurs : 10 perf. + 5 eff.), 24 Go RAM, macOS 26.5, Python 3.13.13. (2) Portable, **Intel Core i5-10210U** 1.6 GHz (4 cœurs / 8 threads), 16 Go RAM.

Méthodologie : `timeit` avec un nombre de répétitions adapté au coût (`number=100` pour César, `number=50` pour Enigma). Sorties `print()` redirigées pour ne pas biaiser la mesure. Moyenne par appel reportée.

Algorithme	Message	Stratégie	M5 Pro	i5-10210U	Ratio	Comb.
Brute César	long (~50 mots)	fréquences	1,17 ms	2 ms	× 1,7	26
Brute César	court (6 mots)	dictionnaire	2,92 ms	16 ms	× 5,5	26
Brute Enigma	long	fréquences	1,12 s	2,07 s	× 1,8	17 576
Brute Enigma	court	dictionnaire	104 ms	985 ms	× 9,5	17 576
Brute César	chaîne vide	dictionnaire	2,76 ms	—	—	26
Brute Enigma	chaîne vide	fréquences (repli)	16,19 ms	—	—	17 576
(réf.) Chargement <code>mots_francais.txt</code>	—	—	2,94 ms	—	—	—

Analyse. César/fréquences (1,17 ms) bat César/dictionnaire (2,92 ms) d'un facteur ~2,5 : pas de chargement de fichier, juste un histogramme et 26 calculs de χ^2 . Le chargement du dictionnaire (~3 ms) est appelé une seule fois par brute-force, son impact relatif diminue donc avec la taille du message. Enigma/fréquences reste plus lent que César/fréquences au-delà du facteur théorique 676 (17 576 / 26) car `enigma_dechiffrer` appelle `cesar.chiffrer/dechiffrer lettre par lettre` (un appel de fonction par caractère), alors que `cesar.dechiffrer` traite le message en une seule passe. Enigma/dictionnaire (court) est ~10× plus rapide qu'Enigma/fréquences (long) : malgré les 17 576 combinaisons identiques, le message court

contient bien moins de lettres à traiter par itération. Sur **chaîne vide**, le brute-force ne descend pas à zéro car toutes les combinaisons sont quand même testées : César reste dominé par le chargement du dictionnaire (~3 ms) et Enigma paye 17 576 itérations à vide (~16 ms, soit ~0,9 µs/combo). Le ratio M5 Pro / i5-10210U varie de 1,7× à 9,5× selon le test : les ratios faibles correspondent aux opérations courtes (<5 ms, dominées par le bruit de mesure), les ratios élevés (5–9×) reflètent la différence réelle de performance brute. L'algorithme se comporte donc de façon cohérente : c'est bien le **matériel** qui explique l'écart.

Optimisation possible (non retenue) — Les 3 clés d'Enigma agissent sur des **positions disjointes** (clé1 sur 0, 3, 6... ; clé2 sur 1, 4, 7... ; clé3 sur 2, 5, 8...). Chacune peut donc être devinée **indépendamment** comme un simple César sur le sous-ensemble correspondant, ramenant le coût de $26^3 = 17\,576$ essais à $3 \times 26 = 78$ essais (gain ~225×). Non retenue ici pour préserver la lisibilité et la cohérence avec la stratégie hybride dictionnaire/fréquences (un mot français est étalé sur les 3 positions, donc inexploitable par sous-ensemble).

4. Distribution des tâches

Membre	Contributions
Nino MINASHVILI	brute-force César, orchestrer, rédaction du rapport
Kilian LEGAVRE	enigma, argparse, README
Victor Elmirzoiev Pradel De Lamaze	brute-force Enigma, tests unitaires, mesures de performance

Méthode collaborative : chaque fonctionnalité développée sur une **branche dédiée** (`fonctions_mise_en_page`, `cesar_dechiffrer`, `orchestre-du-debut`, `enigma`, `branche-des-tests`, `argparse_integration`) puis intégrée à `main` via **pull request** (6 PR mergées). Commits descriptifs en français ; l'historique Git trace l'avancée de chaque membre.